COMPUTER NETWORKS OBSERVATION

Index - HRIDHULA DEVI M 2407013ST

EXT NO.	Date	Experiment Name	Faculty Sign
1	21/7/26	Basic networking commands	28/7/26
2	28/7/26	Study of Network Cables	9 4/8/26
3.	3/8/26	Introduction to cisco Packet Tracer.	10/8/26 10
4	10/8/26	Setup and configure LAN	18/8/26 9
5.	18/8/26	Wireshark	7/9/26 9
6.	7/9/26	Error correction and detection using hamming code.	22/9/26 9
~~7~~	~~21/9/26~~		
7.	22/9/26	NMAP to discover live host (tryhackme)	28/9/26 9
8.	28/9/26	End-End communication at transport layer.	28/9/26 9
9	29/9/26	Flow control at data Link layer	9 29/9/26
10.	29/9/26	Implement your own ping program	9 29/9/26
11.	5/10/26	Subnetting in Cisco Packet Tracer	10 6/10
12.	5/10/26	Raw Sockets	10 6/10
13.	6/10/26	Internetworking with routers	23/10/26 9
14	6/10/26	Subnetting Routing Information Protocol	

21/7/26 EXP. NO. 1 BASIC NETWORKING COMMANDS

Aim:
To study the various network commands used in linux and windows.

(a) Commands in Windows:

(i) arp -a : ARP - address resolution Protocol
It shows the IP address and their corresponding MAC address of devices connected to the network. IP address - given by the routes to find which network its using. Its like name.
MAC address - Its permanent one given by manufacturer. arp -a shows the devices that my computer has recently communicated with. These entries are stored in ARP Cache.

(ii) hostname : Its used to display the name of the computer.

(iii) ipconfig/all : used to display complete network configuration details of computer. It displays: hostname,

hostname, MAC address, IP address, default gateway [entry to outside network].

DHCP (assign the IP address), DNS (connects the web server name to IP address).

(iv) nbtstat -a : used to display NetBIOS name information of computers & used to troubleshoot NetBIOS name resolution problem.

(v) nbstat : used to view the active network connections of computer. It helps us know which devices or servers my computer is communicating with.

(vi) nslookup : used to perform DNS lookups and find the IP address of a website or domain name.

(vii) pathing : used to trace the path between my computer and destination. It also checks each routes on the path for packet loss and helps identifying where network problems

occurs combines ping & tracert.
ping - can I reach the destination?
tracert - which path did I take.

(viii) Ping: Used to check whether another device or website is reachable over network. It sends ICMP echo request packets and waits for echo reply packets to verify the connection between two devices.

(ix) Route: used to display and manage the routing table of computer. It helps configure static routes so that data packets traced through the correct path to reach destination.

(b) Linux Commands:

(i) ip: used to view and manage IP address, network interface and routing information.

(ii) ifconfig: used to configure and troubleshoot network interface. It has now been replaced by ip command.

(iii) mtr: Mtr is traceroute combines ping & traceroute

(iv) tcpdump: Capture tool used to capture and display network packets for network monitoring.

(v) Ping: Whether the destination host is reachable or not over network.

Working:

** ip <option> <object> <command>

Uses:

-> ip address show - shows ip address on server

-> ip address add 192.168.1.254/24 dev enp503

- this is used to assign ip address to interface.

-> ip address del 192.168.1.254/24 dev enp503

- used to delete an ip address in the interface

Result:

Thus the basic networking commands in windows and linux were studied.

→ ip link set eth0 up – alter status of interface by bringing interface eth0

→ ip link set eth0 down – alter the status of the interface eth0.

→ ip route default add via 192.168.1.254 dev eth0 – add default route to the gateway

→ ip route get – display route taken for the ip.

*) mtr <options> hostname/IP

→ mtr google.com – basic command that shows all details.

→ mtr -g google.com – gives all numeric IP addresses instead of hostnames

→ mtr -b google.com – shows numeric IP addresses and hostname

→ mtr -c 10 – used to set the number of pings you want to.

*) tcpdump – used for capturing packets.

→ dnf install -y tcpdump - Installing the command

→ tcpdump -D - This allows users to filter packets based on hosts, protocol, port etc

→ tcpdump -i eth0 - This is used to capture traffic in eth0.

→ tcpdum -i eth0 -c 10 host 8.8.8.8
- this is used to filter traffic from host 8.8.8.8

→ tcpdump -i eth0 src host 8.8.8.8 -
- this is used to capture traffic coming from 8.8.8.8.

→ tcpdum -i eth0 dst host 8.8.8.8
- this is used to capture outbound traffic going to 8.8.8.8.

→ tcpdump -i eth0 port 53
- this is used to capture only DNS port 53 traffic.

→ tcpdump -i eth0 host 8.8.8.8 and port 53
- this is used to capture 8.8.8.8 hosts traffic from port 53.

→ tcpdump -i eth0 -c 10 host www.google.com and port 443 - this is used to capture HTTPS traffic.

→ tcpdump -i eth0 port not 53 and not 25
- this is used to capture all port traffic except port 80 and 25

*) ping google.com

O/P: PING google.com (216.58.206.174) bytes of data
64 bytes from sof02s24-in-f14.1e100.net (216.58.206.174): icmp_seq=1 ttl=56 time=10.7 ms
:
:

*) ping -c 10 google.com - this returns the host pinged 10 times.

* nmcli - If you want to connect a host to network over ethernet, the connection is setting on command line using this command

⇒ nmcli connection show : lists the Network manager connection profiles.

⇒ nmcli connection add con-name <connection-name> ifname <device-name> type ethernet - this is used to add the connection.

⇒ nmcli connection modify "Wired Connection 1" - this is used to rename connection profile.

28/3/20

Result: Thus the basic networking commands in Windows and Linux was studied.

28/4/26

Experiment 2 - Study of Network Cables

Aim: To study different Network Cables

a) understand different type of network cables

There are four major type of cables used in networking

- → unshielded twisted Pair (UTP) cable
- → shielded Twisting Pair (STP) cable
- ⇒ coaxial cable
- → fibre optic cable

Cable type	Category	Maximum Data transmission	Advantages Disadvantages	Application
UTP	Category 3	10 bps	Advantages → Cheaper in cost → easy to install as they have a small diameter Disadvantage → more prone to EMI	10Base-T-Ethernet
	Category 5	Upto 100mbps		Fast ethernet gigabit ethernet
	Category 5e	1Gbps		Faster ethernet gigabit ethernet
STP	Category 6, 6a	10Gbps	Advantages: → shielded → faster → less susceptible to noise Disadvantage → expensive → greater installation effort.	gigabit ethernet 55m Widely used in data centers 10G ethernet
SSTP	Category 7	10Gbps		

Cable type	Category	Maximum Data Transmission	Advantages Disadvantages	Application
Coaxial Cable	RG1-6 RG1-59 RG1-11	10-100 Mbps	Advantages → less loss band width → high band width → versatile → immune to interference Disadvantage → limited distance → cost → size is bulky	speed of signal is 50 m Television network, high speed internet connections
fibre optics Cable	single mode multi mode	100 Gbps	Advantages → high speed → high bandwidth → security Disadvantages → expensive → requires skilled installers	Maximum distance of fibre optics cable is around 100 meters.

Result: Thus the types of cables was studied.

4/8/26

3/8/26

Experiment - 3 - Introduction to Cisco Packet Tracer

Aim:

To study the installation and user interface of Cisco Packet Tracer and analyse the behavior of hub and switch.

Introduction

Cisco Packet Tracer is a network simulation tool used to design, configure and troubleshoot networks without physical hardware. It is available on windows and linux and simulates real networking devices and protocols.

Installation:

→ download Packet tracer from Cisco Networking academy.
→ install it by following setup wizard
→ Launch application after installation.

Procedure:

*) Create two networks
   → 4 PCs connected to Hub
   → 2 PCs connected to switch.

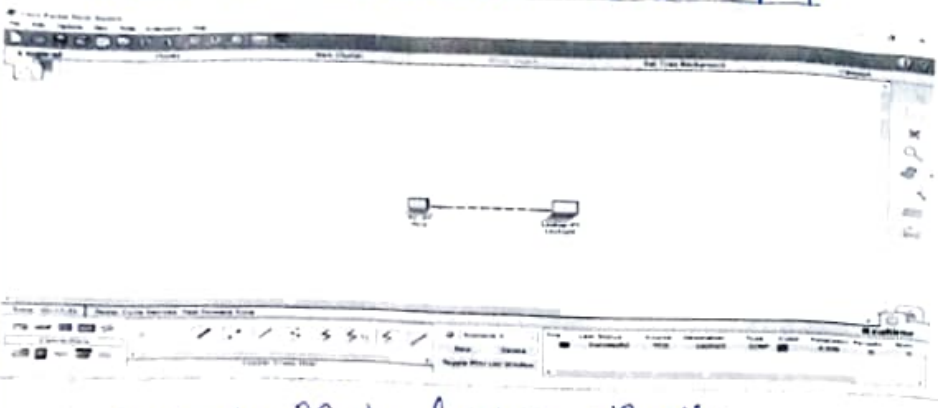
- Connect devices using copper straight through cables
- Assign IP addresses to all PCs
- Send a PDU between 2 PCs
- Observe Packet flow in realtime / simulation mode.
- Compare the working of switch and Hub

Observation:

Hub: Broadcasts data to all connected devices, causing unnecessary traffic

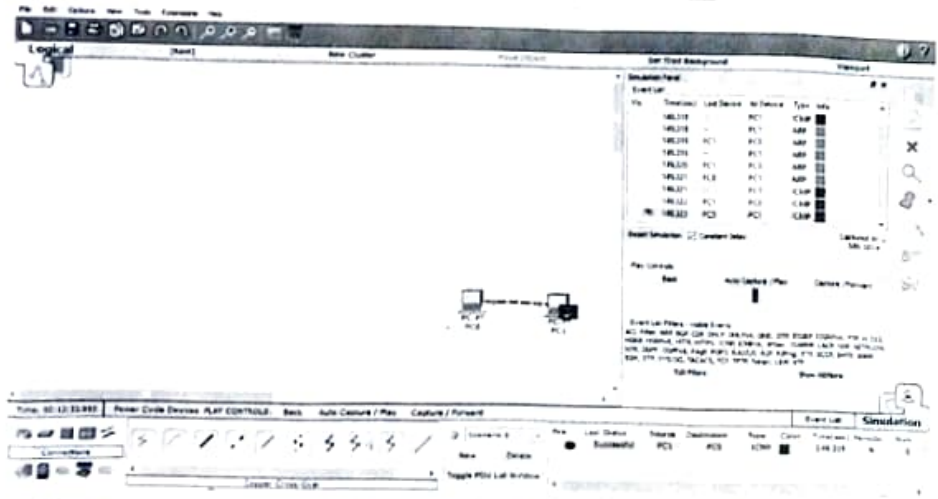
Switch: Sends data only to the destination device using MAC addresses, making communication faster and more efficient.

PEER-PEER Network: PC to Laptop



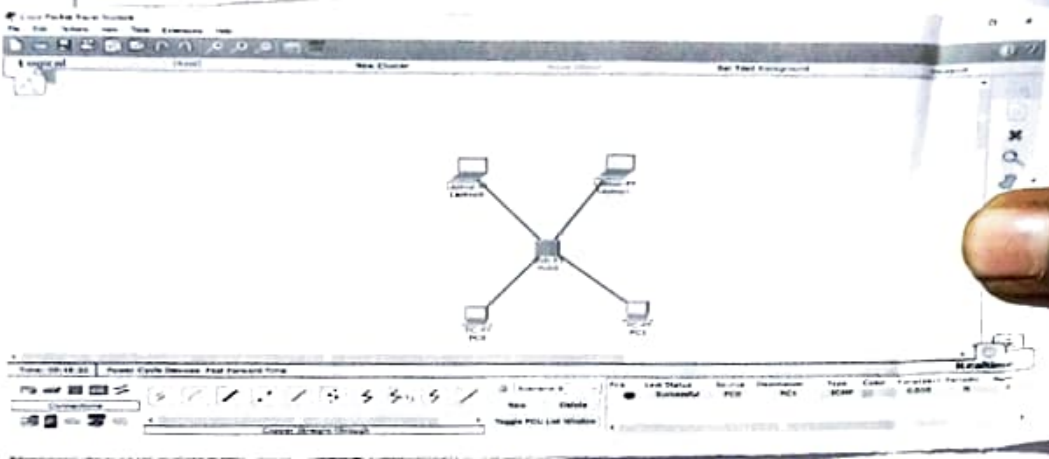
→ Connects PC to laptop directly to share data

Peer-Peer Network : PC to PC



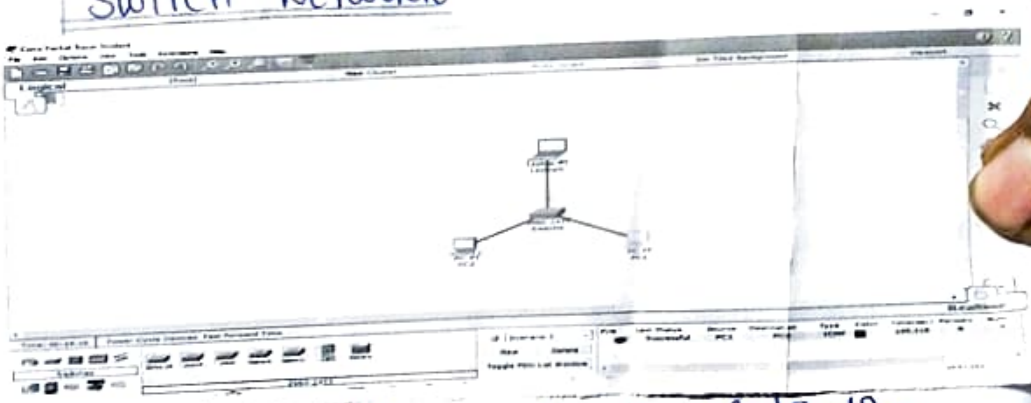
→ Connects 2 PCs to share data directly.

HUB NETWORK:



→ Broadcasts data to all devices that are available

SWITCH NETWORK:



→ Sends data only to destination device. using MAC addresses

→ Makes communication faster.

Result: Thus the experiment was carried out successfully.

16/8/24

10/8/26

Experiment 4 - Setup and configure a LAN using Switch and ethernet cables

LAN:

It stands for local area network which is a group of computers and connected devices linked together in small physical space like home, school etc.

Setup:

1) Connect hardware
   - → place routers in central location near power outlet
   - → connect your modem to the router's WAN or Internet port using an ethernet cable
   - → plug router to power.

2) Connect your device
   - → wired setup → plug one end of an ethernet cable into your computer's network port and other end into one of numbered LAN ports on router.

3) Configure router

→ Type your IP address (192.168.1.1) in to address bar and press enter

→ Login using the default username and password printed on Router.

4) Check connectivity between switch and other machine by using ping command in command prompt of device.

5) Try to access shared folder from other computers of network.

Result:

Wireless LAN using switch and ethernet cables are executed successfully in cisco packet tracer.

10/5/22

18/8/26

Experiment 5 : Wireshark

Aim:

To filter, capture, view, packets in wireshark tool, capture 100 packets from ethernet : IEEE 802.3 LAN interface and save it.

Procedure:

*) Select Local Area Connection in wireshark

*) To capture option

*) select stop capture automatically after 100 packets.

*) Then click start capture

*) Search TCP packets in search bar

*) To see flow graph click statistics flow graph

*) Save packet

1) Create filter to display only TCP/UDP packets, inspects the packets and provide flow graph

*) select LAN connection in wireshark
*) To capture option
*) select stop capture automatically after 100 packets
*) click capture then save

2) Create a filter to display only ARP packets and inspect the packets

*) go to capture option
*) select stop capture automatically after 100 packets
*) Then click start capture
*) search ARP packets in search bar
*) save packets

3) Create a filter to display only HTTP packets and inspect the packets procedure.

*) Select local area LAN in wireshark
*) go to capture option
*) select stop capture after 100 packets.
*) click start, search HTTP packets in search bar.

4) Create a file to display only DNS packets and provide flow graph.
*) Go to capture option.
*) select stop capture automatically after 100 packets
*) Then click start
*) search DNS in search bar
*) To see flow graph click statistics flow graph.

5) Create a filter to display only DHCP packets and inspect packets

*) follow same procedure

*) search DHCP packets in search bar.

Result:

Wireshark on packet capture tool experiment was executed successfully.

16/8/24

7/9/26

Exp-6: Error Correction and Detection With Hamming Code

Aim:

Write a program to implement error correction and detection using hamming code.

Sender:

*) Input to sender file should be text on length program should convert text to binary

*) Apply hamming code concept on binary data and add redundant bits to it.

*) save this output in file called channel

Code:

```
data = input("Enter 4 bits")
d1,d2,d3,d4 = map(int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
Code = [P1, P2, d1, P4, d2, d3, d4]
print("sender hamming code:", "".join(map
        (str, code)))
```

Reciever side:

*) recieve hamming code from sender
*) enter recieved code
*) calculate parity check values
*) find error position using parity values
*) Correct error and display correct code.

Code:

```
r = list(map(int, input("Enter recieved
    7 bits ")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]

error = r1*1 + r2*2 + r4*4

if error == 0:
    print("no error")

else:
    print("Error at position", error)
    r[error-1] ^= 1
    print("Corrected code", ''.join(map(
        str, recieved)))
```

I/P:

Enter 4 bits : 1011

Sender hamming Code : 0110011

O/P: Error at position 2

Corrected code : 0110011

I/P: Enter 4 bits : 0010

Sender hamming code : 0101010

Enter received 7 bits : 0101010

O/P: No error.

Result:

Error detection and correction using hamming code was carried out successfully

22/9/26

Exp 7 : NMAP to discover Live Hosts using NMAP scans on TryHackMe Platform

Aim:

To discover live hosts using NMAP scans (ARP, TCMP, TCP/UDP) on TryHackMe platform

Procedure:

Task 1: Introduction

*) ARP scan: This scan was ARP requests to discover live hosts

*) ICMP scan: to identify live hosts

*) TCP/UDP ping scan: Sends packets to TCP ports and UDP ports to determine live hosts.

Task 2: Subnetworks:

Network segment - It is collection of computers linked together through common "medium"

Subnetwork: It is usually the equivalent of one or more networks segments linked together and configured to utilize the same routes in and IP networks

a) subnets with /16 or /24

Task 3:

1) list:
   - <Machine_IP> <scanme.nmap.org>

2) range
   - 10.11.12.15-20
   - scan 6 IP addresses

3) Subnet
   <machine_IP>/30
   nmap -SL <Targets>

Task 4:

Discovering live hosts:

*) ARP for man in middle attack

*) ICMP from network layer used in DDOS attacks also UDP

*) TCP from transport layer for SYN flooding.

Task 5: Nmap host discovery using ARP

1) When privelaged user scans NMAP uses ARP requests.

2) nmap -PR -sn <Targets>

3) nmap -PR -sn <machine IP>/24

4) nmap -PR -SN <attackbox>/24

Task 6: Namap host discovery using ICMP

*) nmap -PE -sn Target to check host live

*) -PP option in nuts nmap to ICMP timestamp requests

Task 7: Nmap host discovery using TCP/UDP

→ nmap -PS -Sn Target
checks TCP port is open or not

→ -PS21, -PS25 targets ports 21, 22, 23, 24, 25.

→ TCP ACK ping. run as prevelaged user

→ UDP ping - sends UDP packets

→ nmap -PU -Sn Target.

Task 8: Using reverse -DNS lookup

→ Nmap reverses -DNS of internet hosts

→ -R option to query the DNS server even offline hosts.

Task 9: ARP scan, ICMP echo scan, ICMP timestamp scan, ICMP address mask scan, TCP SYN ping

Result: Thus the experiment was carried out successfully.

22/9/25

28/9/26

Exp-8 End-End Communication at transport layer

a)

Aim:

Implement echo client server using TCP/UDP packets

Procedure:

*) Create a socket using python socket programming.

*) Specify the IP address and port number

*) Bind the server socket to the IP address and port.

*) Make the server listen for Client connections.

*) Connect client to server

*) Send and receive message between client and server

*) Display recieved message

*) Close the socket connection.

Code:

TCP- server

```
import socket
server = socket.socket(socket.AF_INET, socket.
                       SOCK_STREAM)
server.bind("127.0.0.1", 5000))
server.listen(1)
print("server is waiting for connection")
Conn. addr = server.accept()
print("Client Connected")

while True:
    data=conn.recv(1024).decode()
    if not data:
        break
    print("Client :", data)
    Conn.send(data.encode())
Conn.close()

server.close()
```

TCP- receiver(client):

```
import socket
Client = socket.socket(socket AF_INET,
socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message")
client.send(message.encode())
reply = client.recv(1024).decode()
```

```
print ("server", reply)
client.close()
```

Terminal - server

```
python tcp-server.py
server is waiting for connection...
client connected
client : Hi
```

Terminal - client

```
python tcp client.py
Enter message: Hi
server : Hi
```

TCP echo is working

Result:

The TCP/UDP packets using echo client server socket program is executed successfully

28/11/24

b)

Aim:

Implement chat client server using TCP packet.

Code:

Chat_server side:

```
import socket
server = socket.socket(socket.AFINET,
         socket.SOCKET_STREAM)
server.bind (("127.0.0.1",6000))
server.listen(1)
print ("waiting for client ...")
conn, addr = server.accept()
print ("Client connected")
while True:
    message= conn.recv(1024).decode()
    if message.lower() == "bye".
        print ("client ended chat")
    break
    print ("Client", message)
    reply = input ("server")
    conn.send (reply.encode())
    if reply.lower()=="bye":
    Conn.close
server.close().
```

Chat - Client side :

```
import socket
client = socket.socket(socket.AF_INET,
         socket.SOCK_STREAM)
client.connect(("127.0.0.1", 6000))
print("connected to server")
while True:
    message = input("client")
    client.send(message.encode())
if message.lower() == "bye":
    break
reply = client.recv(1024).decode()
print("server", reply)
if reply.lower() == "bye":
    break
client.close()
```

Terminal 1

```
python chat_server.py
waiting for client..
client connected
client: Hi
server: Hello
client: How are you?
server: fine
```

Terminal 2

```
python chat_client.py
connected to server
client: Hi
server: Hello
client: How are you?
server: fine
```

Result:

The chat programming using sockets by TCP client and server was implemented successfully.

28/9/22

29/9/26

Exp 9 - Flow control at Data link layer (Sliding Window)

Aim:

To implement flow control at the data link layer using sliding window protocol.

Sliding window Protocol:

flow control technique used to control at the data link layer using sliding window protocol.
Its used to control the number of frames sent before receiving acknowledgements.

Working: Sender

→ Enter window size
→ Enter message
→ Divide the message into frames
→ send frames according to window size.
→

→ Store the frames in sender-buffer
→ wait for acknowledgement
→ If the expected ACK is recieved send next set of frames
→ If nack is recieved, resend the requirement frames

Receiver:
→ Read frames from sender-buffer
→ check frame number
→ If frame no. is correct ; send ACK
else send NACK.

Code:

```
import time
window = int(input("Enter WS"))
message = input("Enter message")
start = 0
while start < len(message):
    print("sender")
    end = start + window
    if end > len(message):
        end = len(message)
sender_buffer = []
```

```
for i in range (start, end):
    frame = (i, message[i])
    sender_buffer.append(frame)
    print("Frame No", i, "Data:",
    message[i])
print("\n sender buffer")
print(sender_buffa)
print("\n waiting for reciever")
time.sleep(1)
print("Reciever")
expected = start
error = False
for frame in sender_buffer:
    frame_no = frame[0]
    data = frame[1]
    print("Recieved frame:",
    frame_no, "Data", data)
    if frame_no == expected:
        expected = expected + 1
    else:
        error = True
        break
```

else error:
    received_buffer = ("Nack", expected)
print ("Transition completed")

I/O: Enter WS : 4
Enter message : HELL

O/P:
Sender
Frame No : 0 Data : H
Frame No : 1 Data : E
Frame No : 2 Data : L
Frame No : 3 Data : L
Sender. Buffer:
[(0,'H'), (1,'E'), (2,'L'), (3,'L')]
Waiting for receiver

Receiver

Received Frame: 0 Data: H
Received Frame: 1 Data: E
Received Frame: 2 Data: L
Received Frame: 3 Data: L

Receiver Buffer:
('ACK', 3)
Sender checking Response
ACK received: 3
Transition completed.

Result:
Thus the experiment was successfully implemented.

29/9/22

Exp - 10 Implement your own Ping Program

Aim:

To implement ping program using UDP client server communication and calculate RTT.

Program:

```
import socket
import time
import sys
# UDP ping server
def
udp_ping_server host = "127.0.0.1",
port = 12000):
server_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
server_socket.bind((host, port))
print(f"[server] Listening on {host}:{port}")
while True:
```

```
msg. client_address = server_socket
recv. form(1034)
print(f"[server] received 'msg.
decode()}' from {client_address}")
server_socket. sento(msg, client_addres
```

UDP ping client

```
def
udp_ping_client(server_host = "127.0.0.1"
server_port = 12000, count=5):
client_socket = socket. socket(socket. AF_INET,
socket. SOCK_DGRAM)
Client. socket. settimeout(1)
for i in range(1, count+1):
    msg = f"Ping {i} {time. time()}
    start = time. time()
    client_socket. sendto(msg. encode())
    (server_host, server_port)
)
try:
    data, _ = client_socket. recvfrom(1024)
    end = time. time()
    rtt = (end - start) * 1000
    print(f" reply from {server_host},
    {server_port}, {server_port}" /".
```

```
f"{data.decode()} | RTT = {rtt:.2f} ms")
except socket.timeout:
        print()
client.close()

run as server or client:
if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv
        [1] == "server":
        udp_ping_server()
    else:
        udp_ping_client()
```

O/P:

server side
[server] listening on
127.0.0.1 : 12000
[server] Received 'ping 1728123456' from
('127.0.0.1', 54321)
...
client side
Reply from 127.0.0.1:12000 | ping 1728123456 | RTT = 0.32ms

Result

Thus the experiment was implemented successfully

5/10/25

Exp-101 Subnetting in Cisco Packet Tracer

Aim:

To implement classless IP subnetting using Cisco Packet Tracer and verify connectivity.

Requirements:

1) Cisco Packet Tracer
2) Routers
3) Switches
4) PC's
5) Ethernet cables.

Procedure:

→ Create new network topology in Cisco Packet Tracer

→ Add routers, switches, PCs and connect using ethernet cables.

→ Divide network into smaller subnets using 127 subnet masks.

→ Configure IP addresses and subnet mask of router interface.

→ Configure IP address, subnet mask and default gateway.

→ Configure required ports.

→ Enable router interface using shut down command

→ Check IP configuration and devices in network.

Result:

Thus the experiment was implemented successfully.

5/10/26

6/10/26

Exp-12 Raw Sockets

Aim:

To implement raw sockets

Procedure:

→ open VS code as administrator
→ paste Do the raw socket programming
→ Run in terminal.

Code:

```
import socket
import struct
HOST = socket.gethostbyname(socket.gethostname())

sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
sock.bind((HOST, 0))
```

```
sock.setsockopt(socket.IPPROTO_IP, socket.IP
                    _HDRINCL, 1)
sock.ioctl(socket.SIO_RCVALL, socket.RCVALL_O)
print("Packet sniffing started...")
print("HOST IP", HOST)
print("Press Ctrl+C to stop\n")
try:
    while True:
        packet, address = sock.recvfrom(65535)
        ip_header = packet[0:20]
        iph = struct.unpack("!BBIHHBBH",
        ip_header)
        source_ip = socket.inet_ntoa(iph[8])
        destination_ip = socket.inet_ntoa(iph[9])
        protocol = iph[6]
        print("===")
        print("Source IP", source_ip)
        print("Destination IP", destination_ip)
        print("Protocol", protocol)
        print("Packet Size", len(packet))
        print("===")
```

```
except KeyboardInterrupt:
    print ("\n Sniffing stopped")

finally:
    sock.ioctl(sock.SIO_RCVALL, socket.
    RCVALL_OFF)
    sock.close().
```

Result:

Thus the experiment was implemented successfully.

6/10/22

10/26 Exp: 13

Internetworking with Routers

Aim:

To design and configure an internetwork using a router in Cisco packet Tracer

Procedure

→ Place one router and 2 PC's
→ Connect both PC's to router using straight Cables
→ Assign IP addresses and subnet masks to router interface and PC's
→ Configure the default gateway for each PC
→ Enable router interface using no shutdown
→ Test connectivity using ping command

Router Configuration:

```
test
enable
configure terminal
interface FastEthernet 0/0
ip address 192.168.10.1
255.255.255.0
no shutdown
exit
```

Observation:

The router connects 2 different networks and transfers packets

Result:

Thus the experiment was successfully executed.

6 Simulate RIP using CISCO Packet Tracer.

Aim

To simulate RIP using CISCO Packet Tracer.

Procedure

* Create the topology with routers and 2 PCs
* Connect the device using suitable Cables.
* Assign the given IP address to both PCs
* Configure Fastethernet and serial interface of all routers
* Enable interface using no shutdown.
* Let clock rate & 4000 and bandwidth 64 on DE serial interface.
* Enable RIP on all routers using router rip
* Add directly connected networks using the network command.
* Test connectivity using ping 20.0.0.2
* use tracert 20.0.0.2 to verify the selected route.
* Disconnect the direct Router 0 - Router 2 link and use tracert again to verify RIP selects the alternate route

→ set main and backup routes using different administrative distance value
→ test connectivity
→ use tracer to verify routing packets.

Result:
Thus the experiment was carried out successfully.